

T04 — Sistema di Tipi, Primitive e Reference

Tipi Primitivi vs Reference, Casting, Wrapper, Operatori e Analisi di Example.java

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

Con questo blocco entriamo nel vivo della semantica del linguaggio: analizziamo come la JVM gestisce i tipi, la memoria (Stack vs Heap), i puntatori/reference, gli operatori logici e lo scope, confrontando la teoria di [T04 - Types and Objects.pdf](#) con il primo file di codice ufficiale del docente: [Example.java](#).

0.0.1 1. Teoria: Concetti Teorici dalle Slide

A. Tipizzazione dei Linguaggi e il “Contratto” di Tipo (Slide 1–7)

- **Definizione formale informale di Tipo:** Un tipo è una coppia formata da un **insieme di valori ammissibili** e da un **insieme di operazioni consentite** su di essi.
 - Es. `int`: valori da -2^{31} a $2^{31} - 1$, operazioni `+`, `-`, `*`, `/`, `%`.
 - Il tipo definisce un **contratto**: vincola cosa una variabile può contenere e in quali espressioni può comparire.
- **Weakly Typed (es. Python, JavaScript) vs Strongly Typed (Java):**
 - Nei linguaggi *weakly typed*, le variabili sono contenitori generici slegati dal tipo: una variabile `x` può contenere un intero, poi una stringa, poi una tupla.
 - Nei linguaggi *strongly typed* come Java, ogni variabile richiede una dichiarazione esplicita e immutabile di tipo a tempo di compilazione (`int x;`). Assegnare un tipo incompatibile genera un **Compile-Time Error**.
 - *Meme di JavaScript a lezione:*
 - * `"11" + 1` $\Rightarrow$ `"111"` (coercizione implicita a stringa e concatenazione).
 - * `"11" - 1` $\Rightarrow$ `10` (coercizione implicita a numero e sottrazione numerica).
 - * In Java queste ambiguità ed errori silenziosi sono impossibili: il compilatore blocca ogni operazione non conforme prima dell'esecuzione.

B. I Tipi Primitivi in Java e le Conversioni (Slide 8–13) Java dispone di **8 tipi primitivi** (allocati direttamente per valore nello Stack o inline negli oggetti, non sono oggetti):

Tipo Primitivo	Dimensione	Range di Valori	Default (campi)
byte	8 bit (1 byte)	da -128 a $+127$ ($-2^7 \dots 2^7 - 1$)	0
short	16 bit (2 byte)	da -32.768 a $+32.767$ ($-2^{15} \dots 2^{15} - 1$)	0
int	32 bit (4 byte)	da $-2.147.483.648$ a $+2.147.483.647$ ($-2^{31} \dots 2^{31} - 1$)	0
long	64 bit (8 byte)	da -2^{63} a $+2^{63} - 1$ (suffisso L o l)	0L
float	32 bit IEEE 754	Precisione singola (suffisso obbligatorio f o F)	0.0f
double	64 bit IEEE 754	Precisione doppia (default per numeri con la virgola)	0.0d
char	16 bit (2 byte)	da 0 a 65.535 (Unicode UTF-16, unsigned)	'\u0000' (NUL)
boolean	1 bit logico	true oppure false	false

- **Aritmetica Modulare e Overflow degli int:**

- In Java non viene lanciata alcuna eccezione se un calcolo intero supera il limite massimo: avviene il **wrap-around** secondo l'aritmetica del complemento a due a 32 bit.
- Se $a = 2147483647$ ($2^{31} - 1$), l'operazione $a + 1$ restituisce -2147483648 (-2^{31}).

- **Conversioni di Tipo (Casting):**

- **Widening (Allargamento / Implicito):** da un tipo più stretto a uno più ampio (nessuna perdita d'ordine di grandezza):

byte $\longrightarrow$ short $\longrightarrow$ int $\longrightarrow$ long $\longrightarrow$ float $\longrightarrow$ double

Es: float f = 3; converte automaticamente int 3 in 3.0f.

- **Narrowing (Restringimento / Esplicito obbligatorio):** da un tipo più ampio a uno più stretto, richiede il cast sintattico (tipo) perché comporta potenziale troncamento o perdita di informazione:
Es: int i = (int)3.2; scarta la parte decimale e memorizza 3.

- **I Caratteri sono Numeri Interi Unsigned a 16 bit:**

- Un char memorizza il codice numerico Unicode (UTF-16 code unit).
- 'V', (char)86 e '\u0056' sono tre rappresentazioni letterali dello **stesso identico dato binario** (valore decimale 86).

- **Tipi Non-Primitivi (Reference Types) e la Classe String:**

- Tutto ciò che non è un tipo primitivo è un **oggetto**.
- String è una classe immutabile. L'operatore + tra una stringa e qualsiasi altro tipo (primitivo o oggetto) applica automaticamente l'overloading di concatenazione, invocando internamente la conversione a stringa.

C. Modello di Memoria, Puntatori e Garbage Collection (Slide 14–21)

- **Assenza di Variabili Globali:** In Java non esiste lo scope globale in stile C. Ogni dato o funzione appartiene a una classe o a un'istanza.
- **La Verità sui Puntatori in Java:**
 - «*Java does have pointers, but there is no pointer arithmetic*»: ogni variabile di tipo non primitivo è tecnicamente un **puntatore/riferimento** (*reference*) a un blocco di memoria nello **Heap**.
 - La JVM protegge la memoria: il programmatore non può vedere l'indirizzo fisico, né fare `ptr++` o dereferenziare offset arbitrari.
 - Il passaggio dei parametri ai metodi in Java è **sempre rigorosamente per valore** (*pass-by-value*): per gli oggetti, viene copiato per valore il **riferimento** (l'indirizzo logico).
- **Rappresentazione di un Reference:**
 - Una variabile d'istanza non inizializzata contiene `null` (es. `Vehicle@null`).
 - Quando viene invocato `new Vehicle()`, la JVM alloca l'oggetto nello Heap e restituisce l'indirizzo: `myCar` conterrà un valore del tipo `Vehicle@25c7f37d`.
- **Errori di Memoria e NullPointerException (NPE):**
 - Se si tenta di invocare un metodo o accedere a un campo su una reference che vale `null` (es. `myCar2.setFrameNumber(5)`), la JVM solleva una **NullPointerException** a runtime, interrompendo l'esecuzione.
- **Perché i programmatori scrivono `myCar = null`?:**
 - L'assegnamento a `null` rimuove il puntatore verso l'oggetto nello Heap. Se nessun'altra reference punta a quell'area di memoria, l'oggetto diventa **irraggiungibile** (*unreachable*) ed eleggibile per la **Garbage Collection (GC)**, consentendo alla JVM di bonificare la memoria.

0.0.2 2. Codice: Analisi Dettagliata del Codice: `Example.java`

Analizziamo riga per riga il sorgente della Lezione 04, evidenziando tutte le trappole e i dettagli d'esame:

```

1 public class Example {
2     static float f1;
3     static float f2 = 3.4f;
4     static String s1;
5     static final double PI = 3.14;
6
7     public static void main(String[] args) {
8         int i1 = 6;
9         int i2;
10        char c1 = 'V';
11        char c2 = 86;
12        char c3 = '\u0056';
13        String s2;
14
15        System.out.println("i1: " + i1);
16        // System.out.println("i2: " + i2); // compile-time error
17        System.out.println("f1: " + f1);
18        System.out.println("f2: " + f2);
19        System.out.println("c1: " + c1 + " c2: " + c2 + " c3: " + c3);
20        System.out.println("s1: " + s1);
21        /*
22         System.out.println("s2: " + s2); // compile-time error
23         System.out.println(s1.toString()); // run-time error
24         PI = 3.0; // compile-time error
25        */
    }
}

```

```

26     int k = 0;
27     System.out.println(true | k++ == 0);
28     System.out.println("k (standard eval): " + k);
29     k = 0;
30     System.out.println(true || k++ == 0);
31     System.out.println("k (short-circuit eval): " + k);
32     //
33     int n = 4;
34     n ++;
35     // int n = 8; // compile-time error
36     {
37         int m = 7;
38         m ++;
39     }
40     {
41         int m = 8;
42         m --;
43     }
44     System.out.println("n: " + n);
45     // System.out.println("m: " + m); // compile-time error
46 }
47 }

```

Analisi: Le Finezze Implementative Spiegate Riga per Riga:

1. Variabili Statiche/Campi vs Variabili Locali (*Definite Assignment Analysis*):

- `static float f1;` e `static String s1;` sono campi a livello di classe. **La JVM li inizializza SEMPRE automaticamente al loro default:** `f1` diventa `0.0f` e `s1` diventa `null`.
- `int i2;` e `String s2;` sono variabili locali allocate nel frame dello **Stack**. **Le variabili locali NON hanno valori di default.**
- Se tentiamo di leggere `i2` o `s2` prima di aver assegnato loro un valore (`System.out.println(i2)`), il compilatore Java blocca la compilazione con l'errore: `variable i2 might not have been initialized`.

2. Suffisso dei Floating Point (3.4f):

- `static float f2 = 3.4f;` il letterale `3.4` senza suffissi è un `double` a 64 bit. Scrivere `float f = 3.4;` genera errore di compilazione (*loss of precision*). Il suffisso `f` forza il letterale a 32 bit.

3. Costanti con `final`:

- `static final double PI = 3.14;` la keyword `final` rende la variabile a sola lettura dopo l'inizializzazione. Il tentativo di riassegnamento `PI = 3.0;` viene respinto dal compilatore (`cannot assign a value to final variable PI`). Per convenzione Java, le costanti `static final` si scrivono in MAIUSCOLO_SNAKE_CASE.

4. I 3 modi di rappresentare un `char`:

- `c1 = 'V';` letterale carattere.
- `c2 = 86;` valore intero decimale ASCII/Unicode per la 'V'.
- `c3 = '\u0056';` sequenza di escape Unicode a 16 bit in esadecimale ($5 \times 16 + 6 = 86$).
- Stampati a video, producono tutti e tre il carattere `V`.

5. Concatenazione di `null` vs `NullPointerException`:

- `System.out.println("s1: " + s1);` stampa `"s1: null"`. L'operatore `+` converte in sicurezza il riferimento nullo nella stringa letterale `"null"`.
- `System.out.println(s1.toString());` genera invece un **Run-Time Error** (`NullPointerException`): non è possibile dereferenziare un puntatore nullo per invocarvi un metodo d'istanza.

6. Operatori Logici Standard (`|`, `&`) vs Corto-Circuito (*Short-Circuit* `||`, `&&`) e Side Effects:

- Caso `true | k++ == 0`:
 - L'operatore singolo `|` (non a corto circuito) **valuta SEMPRE entrambi gli operandi**.
 - Anche se la parte sinistra è già `true`, la parte destra `k++ == 0` viene valutata: `k` viene confrontato con `0` (dando `true`) e poi incrementato di 1.
 - Stampa a video: `true`, seguito da `k` (standard eval): 1.
- Caso `true || k++ == 0`:
 - L'operatore doppio `||` (a corto circuito) verifica la parte sinistra: poiché è `true`, il risultato globale dell'or logico è già matematicamente certo (`true`).
 - La parte destra viene **completamente ignorata e mai valutata!**
 - Di conseguenza, `k++` **non viene eseguito**.
 - Stampa a video: `true`, seguito da `k` (short-circuit eval): 0.
- *Importanza pratica*: lo short-circuit `&&` è l'idioma fondamentale per evitare crash su `null`:

```
1 if (s != null && s.length() > 0) { ... } // Se s è null, non chiama mai s.length()!
```

7. Scope delle Variabili e Divieto di Shadowing Locale:

- In Java, il ciclo di vita e la visibilità delle variabili sono delimitati dalle graffe `{ ... }`.
- `int n = 4; n++;` dichiara `n` nello scope del metodo. Scrivere `int n = 8;` nello stesso scope o in un sottoblocco interno genera l'errore: `Variable 'n' is already defined in the scope`. In Java le variabili locali non possono oscurare altre variabili locali omonime.
- Blocchi anonimi indipendenti:

```
1 { int m = 7; m++; }
2 { int m = 8; m--; }
```

La variabile `m` del primo blocco viene distrutta alla chiusura della prima graffa. La seconda `m` è una nuova variabile in uno scope disgiunto. All'esterno dei blocchi, `m` non esiste (`cannot find symbol variable m`).

0.0.3 3. Test: Output Esatto di Esecuzione

Eseguendo `Example.java` da terminale o IntelliJ:

```
1 i1: 6
2 f1: 0.0
3 f2: 3.4
4 c1: V c2: V c3: V
5 s1: null
6 true
7 k (standard eval): 1
8 true
9 k (short-circuit eval): 0
10 n: 5
```

Nota di Studio

Con la **Lezione 4 / T04** abbiamo chiarito le basi di tipo, stack/heap e operatori.

Il prossimo blocco raggruppa le **Lezioni 05-06 / Slide T05 (Java Basic Syntax) e T06 (Java Classes and Objects)**, dove nasce il progetto incrementale centrale del corso: - Struttura della prima classe ad oggetti: `SimpleDate` (`Date.java` e `MainDate.java`). - La classe sperimentale `StringPlayground.java`. -

La prima validazione delle date: algoritmo dei giorni del mese e calcolo degli anni bisestili secondo il calendario Gregoriano. - Sintassi Java: costrutti di controllo (`if-else`, `switch`, `for`, `while`, `do-while`), etichette (*labeled break/continue*). - Metodi, parametri, ritorno e la keyword `this`. - La classe `MainDate`: test delle istanze, istanziamento con `new` e stampa dello stato.

Dammi conferma per aprire le Lezioni 05-06 / T05-T06 e analizzare la prima versione di `SimpleDate` e `StringPlayground`!